Advanced Computer Architecture Midterm Presentation

Student: Guan-Ting Wu



國立清華大學
NATIONAL TSING HUA UNIVERSITY

Workload

- EfficientNet-Lite0:
  - R. Liu, “Higher accuracy on vision models with EfficientNet-Lite,” TensorFlow Blog, Mar. 16, 2020. [Online].
  - Available: https://github.com/tensorflow/tpu/tree/master/models/official/efficientnet/lite

Workload

- Reason
  - Fit the Spec：4.7 M Parameters / ~0.39 GMAC
  - Officially provides TFLite model with integer-only quantization.
  - ImageNet Top-1: 75.1% FP32 / 74.4% INT8
    - I ran it on 50000 images of image-1K dataset.

Baseline

- Implement TFLite reference into an end-to-end C model for baseline.

- Profiling

Metric	Value	Note
Code Size (.text)	4,408 bytes	-
Dynamic Inst.	4,449,648,842	Total executed instructions (~4.45B)
Memory Inst.	1,083,684,372	~24.35% of total instructions
Estimated MACs	385,187,552	-

Optimization – B1 for Conv

- Observe
  - 1 3x3 filter Conv
  - 32 1x1 filter Conv (to be optimized)

Scope / Function	Dynamic Inst.	Inst. %	Estimated MACs	MAC %
Entire Model (Total)	**4,449,648,842**	**100.00%**	**385,187,552**	**100.00%**
Conv	3,242,185,190	72.86%	349,375,488	90.70%
Depthwise Conv	921,897,095	20.72%	34,532,064	8.96%
Output Scaling	263,126,439	5.91%	—	—

Optimization – B1

- Implement 1
  - Simplify the general kernel of TFLite in B0 into a fast path for 1x1 filter Conv.
  - E.g. Skip the filter loops, pad, etc.

Optimization – B1

- Implement 2
  - In 1x1 filter Conv, the main path doing MAC can be simplified.

MAC Math

```
acc = sum(filter_q * (input_q - input_zp)) + bias_q
                              |
                              V
acc = sum(filter_q * input_q) + (bias_q - sum(filter_q) * input_zp)
                              |
                              V
acc = sum(filter_q * input_q) + pre_calculated_bias
```

Estimated Instruction counts of MAC of 1x1 Conv

```
1 lb input
1 lb filter
1 addi filter pointer
1 addw input, input_offset
1 mulw
1 addi input pointer
1 addw accumulator
1 branch
------------------------
8 instructions / MAC
```



```
1 lb input
1 lb filter
1 addi filter pointer
1 mulw
1 addi input pointer
1 addw accumulator
1 branch
------------------------
7 instructions / MAC
```

Optimization – B1

- Implement 3
  - 4 output-channel accumulators can be calculated continuously.
  - Increase register utility

MAC of 1x1 Conv

```
1 lb input
1 lb filter
1 addi filter pointer
1 mulw
1 addi input pointer
1 addw accumulator
1 branch
------------------------
7 instructions / MAC
```



```
1 input load
4 weight loads
4 multiplies
4 accumulator adds
1 input-pointer update
4 weight-pointer updates
1 loop branch
------------------------
19 instructions / 4 MACs
= 4.75 instructions/MAC
```

Optimization – B1

- Result
  - Instruction reduction: 30.4%

B0

Scope / Function	Dynamic Inst.	Inst. %	Estimated MACs	MAC %
Entire Model (Total)	**4,449,648,842**	**100.00%**	**385,187,552**	**100.00%**
Conv	3,242,185,190	72.86%	349,375,488	90.70%
Depthwise Conv	921,897,095	20.72%	34,532,064	8.96%
Output Scaling	263,126,439	5.91%	—	—

B1

Scope / Function	Dynamic Inst.	Inst. %	Estimated MACs	MAC %
Entire Model (Total)	**3,096,086,990**	**100.00%**	**385,187,552**	**100.00%**
Conv	1,888,623,338	61.00%	349,375,488	90.70%
Depthwise Conv	921,897,095	29.78%	34,532,064	8.96%
Output Scaling	263,126,439	8.50%	—	—

Optimization – X1 for Conv

- Implement
  - Xqmac8 instruction.
  - Make Input & weight 8-byte aligned -> ld load 8 elements in one cycle.

Xqmac8 Contract

```
xqmac8 rd, rs1, rs2

// rd is both input & output
rd = INT32(rd)
     + INT8(rs1[ 7: 0]) * INT8(rs2[ 7: 0])
     + ...
     + INT8(rs1[63:56]) * INT8(rs2[63:56])
```

MAC of 1x1 Conv

```
1 input load
4 weight loads
4 multiplies
4 accumulator adds
1 input-pointer update
4 weight-pointer updates
1 loop branch
-----------------------
19 instructions / 4 MACs
= 4.75 instructions/MAC
```



```
1 packed input load
4 packed filter loads
4 Xqmac8
5 pointer updates
1 loop branch
--------------------
15 instructions / 32 MACs
= 0.47 instruction/MAC
```

Optimization – X1

- Note:
  - Design software (B1) & hardware (X1) together
  - Xqmac8 work more beautifully.

Software design in B1

```
acc = sum(filter_q * (input_q - input_zp)) + bias_q
                              |
                              V
acc = sum(filter_q * input_q) + (bias_q - sum(filter_q) * input_zp)
                              |
                              V
acc = sum(filter_q * input_q) + pre_calculated_bias
```

Xqmac8 Contract

```
xqmac8 rd, rs1, rs2

// rd is both input & output
rd = INT32(rd)
     + INT8(rs1[ 7: 0]) * INT8(rs2[ 7: 0])
     + ...
     + INT8(rs1[63:56]) * INT8(rs2[63:56])
```

Optimization – X1

- Result
  - Instruction reduction: 46.8%

B1

Scope / Function	Dynamic Inst.	Inst. %	Estimated MACs	MAC %
Entire Model (Total)	**3,096,086,990**	**100.00%**	**385,187,552**	**100.00%**
Conv	1,888,623,338	61.00%	349,375,488	90.70%
Depthwise Conv	921,897,095	29.78%	34,532,064	8.96%
Output Scaling	263,126,439	8.50%	—	—

X1

Scope / Function	Dynamic Inst.	Inst. %	Estimated MACs	MAC %
Entire Model (Total)	**1,647,729,506**	**100.00%**	**385,187,552**	**100.00%**
Depthwise Conv	921,897,095	55.95%	34,532,064	8.96%
Conv	440,265,854	26.72%	349,375,488	90.70%
Output Scaling	263,126,439	15.97%	—	—

Optimization – B2 for Depthwise Conv

- Observe
  - The addressing & control instructions may be important at Depthwise Conv.

Scope / Function	Dynamic Inst.	Inst. %	Estimated MACs	MAC %
Entire Model (Total)	**4,449,648,842**	**100.00%**	**385,187,552**	**100.00%**
Conv	3,242,185,190	72.86%	349,375,488	90.70%
Depthwise Conv	921,897,095	20.72%	34,532,064	8.96%
Output Scaling	263,126,439	5.91%	—	—

Optimization – B2

- Implement 1
  - Move the determination of filter ranges outside channel loop.

```
loop(output pixel)
  loop(channel)
    loop(filter_y)
      calculate input_y
      determin y boundary
      loop(filter_x)
        calculate input_x
        determin x boundary
        count input/filter index
        MAC
```

→

```
loop(output pixel)
  determin x boundary
  determin y boundary
  loop(channel)
    loop(filter_y, whithin y boundary)
      calculate input_y
      loop(filter_x, whithin x boundary)
        calculate input_x
        count input/filter index
        MAC
```

  - Walking pointer, calculate less in inner loop

```
input_index = ((B * H + y) * W + x) * C + c;
```

→

```
input_index += c;
```

Optimization – B2

- Implement 2
  - 4 output channels for MAC
  - Reduce loop instructions & pointer updates

inner loop of Depthwise conv

```
1 input load
1 filter load
1 input-offset add
1 multiply
1 accumulator add
2 pointer updates
1 loop branch
-----------------
8 instructions / MAC
```



```
4 input loads
4 filter loads
4 input-offset adds
4 multiplies
4 accumulator adds
2 pointer updates
1 loop branch
--------------------
23 instructions / 4 MACs
= 5.75 instructions / MAC
```

Optimization – B2

- Result
  - Instruction reduction: 40.3%

X1

Scope / Function	Dynamic Inst.	Inst. %	Estimated MACs	MAC %
Entire Model (Total)	**1,647,729,506**	**100.00%**	**385,187,552**	**100.00%**
Depthwise Conv	921,897,095	55.95%	34,532,064	8.96%
Conv	440,265,854	26.72%	349,375,488	90.70%
Output Scaling	263,126,439	15.97%	—	—

B2

Scope / Function	Dynamic Inst.	Inst. %	Estimated MACs	MAC %
Entire Model (Total)	**983,811,822**	**100.00%**	**385,187,552**	**100.00%**
Conv	440,265,854	44.75%	349,375,488	90.70%
Depthwise Conv	257,979,411	26.22%	34,532,064	8.96%
Output Scaling	263,126,439	26.75%	—	—

Optimization – B3 for Requantization

- Observe
  - Constraints below allows me to simplify the general requantization kernel of TFLite.
    - All multipliers > 0
    - -29 < all shifts < -2
    - activation_min = -128
    - activation_max = 127

Optimization – B3

- Implement
  - Simplify controls for a less-general fast path
  - Modify rounding paths

ex: Round to nearest, ties toward +∞

TFLite

```
nudge = product >= 0 ? (1LL << 30) : (1 - (1LL << 30));
q31 = (product + nudge) / (1LL << 31);
```



Me

```
q31 = (int32_t)((product + (INT64_C(1) << 30)) >> 31);
```

Optimization – B3

- Result
  - Instruction reduction: 13.4%

B2

Scope / Function	Dynamic Inst.	Inst. %
Entire Model (Total)	**983,811,822**	**100.00%**
Conv	440,265,854	44.75%
Depthwise Conv	257,979,411	26.22%

B3

Scope / Function	Dynamic Inst.	Inst. %
Entire Model (Total)	**851,551,446**	**100.00%**
Conv	506,111,566	59.43%
Depthwise Conv	299,608,208	35.18%

Optimization – X3 for Requantization

- Observe
  - Not sure how difficult the CVWally implantation will be.
  - For not introducing complicated hardware, I only added a simple **Xclip8** instruction here.
  - More complex requantization instructions will be considered with RTL later.

Optimization – X3

- Implement
  - Xclip8 comparator + mux in RTL

TFLite C clip operation

```
if value > 127:
    result = 127
else if value < -128:
    result = -128
else:
    result = value
```



Xclip8 Contract

```
if value[31:7] == all_1 or value[31:7] == all_0:
    result32 = value[31:0]
else if value[31] == 0:
    result32 = 0x0000007f
else:
    result32 = 0xffffff80
```

Optimization – X3

- Result
  - Instruction reduction: 4.56%

B3

Scope / Function	Dynamic Inst.	Inst. %
Entire Model (Total)	**851,551,446**	**100.00%**
Conv	506,111,566	59.43%
Depthwise Conv	299,608,208	35.18%

X3

Scope / Function	Dynamic Inst.	Inst. %
Entire Model (Total)	**812,700,880**	**100.00%**
Conv	478,505,419	58.88%
Depthwise Conv	288,363,789	35.48%

Conclusion

- Result
  - 5.5x with only Software & Xqmac8 & Xclip8 optimization.

Version	Retired instructions	Reduction vs B0	MACs / instruction	.text bytes	Xqmac8	Xclip8
B0	4,449,648,842	0.00%	0.086566	4,408	0	0
B1	3,096,086,990	30.42%	0.124411	5,012	0	0
X1	1,647,729,506	62.97%	0.233769	5,024	42,317,184	0
B2	983,811,822	77.89%	0.391526	5,860	42,317,184	0
B3	851,551,446	80.86%	0.452336	6,436	42,317,184	0
X3	812,700,880	81.74%	0.473960	6,150	42,317,184	6,737,696

Thank You